

JavaScript Event Loop & Async Architecture – Complete Training Guide

Task 1 – Execution Order

Synchronous code runs first, followed by microtasks (Promises), then macrotasks (timers).

```
Call Stack
  |
  v
console.log
  |
Microtask Queue (Promise)
  |
Macrotask Queue (setTimeout)
```

Task 2 – UI Freeze

Long synchronous loops block the main thread. Async code cannot interrupt execution.

```
User Action
  |
Main Thread (CPU loop)
  |
UI Frozen
```

Task 3 – Timer Drift

setTimeout guarantees minimum delay only. Blocking causes late execution.

```
Timer Registered
  |
Event Loop Blocked
  |
Callback Delayed
```

Task 4 – Callback Queue Mapping

Microtasks execute before macrotasks once the call stack is empty.

```
API Call -> Promise -> Microtask
I/O Event -> Macrotask
```

Task 5 – Promise Chain

Promise chains fully drain before any timers execute.

```
.then()
.then()
-- drained --
setTimeout
```

Task 6 – Async forEach Bug

forEach does not await async callbacks. Use for...of or concurrency control.

```
forEach -> Parallel (unsafe)
for...of -> Sequential (safe)
```

Task 7 – Notification Storm

Large fan-out causes lag and throttling. Batch requests safely.

```
Users
|
Batch Queue
|
Provider
```

Task 8 – API Waterfall

Parallelize independent async calls using Promise.all.

```
Sequential: A -> B -> C
Parallel: A | B | C
```

Task 9 – Race Condition

Async syntax does not prevent data races. Use DB-level locking.

```
User A ---->
                DB Lock
User B ---->
```

Task 10 – Event Loop Lag

Measure timer delay to detect blocking and performance degradation.

```
Expected 0ms
Actual 120ms
```

Task 11 – Microtask Starvation

Recursive microtasks prevent rendering and macrotasks.

```
Microtask
|
Microtask
|
(No Render)
```

Task 12 – CPU vs I/O

CPU-heavy work belongs in workers; I/O belongs to event loop.

```
CPU -> Workers
I/O -> Event Loop
```

Task 13 – Worker Threads

Offload heavy computation to prevent API latency.

```
Main Thread -> Worker Pool
```

Task 14 – Retry Storm

Immediate retries cause cascading failures. Use backoff + jitter.

```
Retry  
Retry+Delay  
Retry+Random
```

Task 15 – Streams vs Buffers

Streams prevent memory exhaustion for large files.

```
File -> Stream -> Client
```

Task 16 – Flash Sale Spike

Queues, rate limits, and circuit breakers protect systems.

```
Users -> Queue -> Services
```

Task 17 – Promise Memory Leak

Unresolved promises retain memory indefinitely.

```
Promise Created  
Never Resolved
```

Task 18 – Rendering Jank

Rendering occurs between macrotasks, not microtasks.

```
.then() chain  
(No Paint)
```

Task 19 – Priority Scheduling

Critical tasks require separate queues.

```
Payments -> High Priority  
Analytics -> Low Priority
```

Task 20 – Observability

Measure event loop lag, percentiles, and throughput.

```
Metrics -> Alerts -> Prevention
```